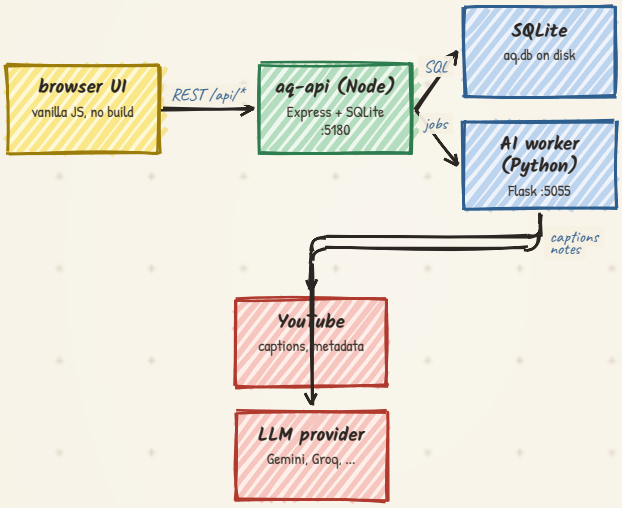
a developer's own field notebook —

AQ Learning

lecture playlists, watch progress and AI hand-drawn notes - the whole system

UI + Node API + Python worker · en



browser -> Node (state) -> Python (AI) -> the internet

what's inside — read top to bottom

2. architecture, the 30,000ft map
3. tech stack
4. repo layout
5. data layer
6. request flow: get notes
7. watch & resume flow
8. notes pipeline in Python
9. ingest.py: transcripts
10. llm.py & prompts.py
11. notes_schema.py: contract
12. pdf.py: the notebook
13. Flask server: the worker
14. python files, one line each
15. UI core files
16. UI feature modules
17. api.js: the sync adapter
18. aq-api: the Node backend
19. playlist import flow
20. progress calculation
21. security
22. run & deploy
23. tests & quality
24. limits & future scope

Node owns state, Python owns the slow AI work!

★ *recall question*

Q: How do I use this book?

A: After each page, close it and explain the page out loud in 30 seconds. Every page ends with one recall question.

Three small programs: a vanilla-JS UI, a Node + SQLite app backend and a Python AI worker. Every file, data store and request, mapped end to end.

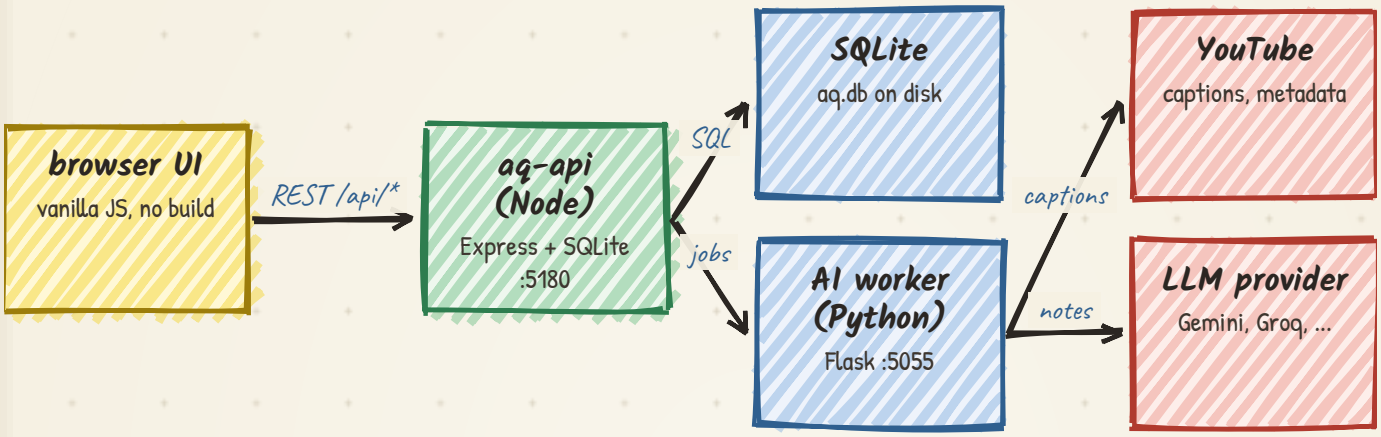
— field notes, Oct 2026

2. architecture, the 30,000ft map

— three programs, one job each

AQ Learning turns YouTube lectures into playlists you can watch in order, with resume-where-you-left-off progress and hand-drawn study notes. It is ***three small programs***, not one big one, and each has exactly one job.

the big picture



browser -> Node (state) -> Python (AI) -> the internet

- ***lecture-notes-ui*** - vanilla JS single page: playlists, player, progress bars, notes viewer. No build step.
- ***aq-api*** (Node + Express + SQLite, :5180) - the application backend: users, playlists, videos, watch progress, notes records cached by video_id. Serves the UI too.
- ***lecture-notes-ai*** (Python + Flask, :5055) - the AI/media worker: YouTube transcript, LLM notes, PDF. Knows nothing about users.

note to self

Rule of thumb: Node owns state and identity, Python owns slow, expensive, stateless work. The UI only ever talks to one door (:5180); Node calls Python on 127.0.0.1.

★ ***to remember***

→ *Three programs, one job each: UI shows, Node remembers, Python generates.*

→ *The UI has one front door (:5180); Python is an internal worker on localhost.*

→ *Node is the place future auth (OAuth) plugs in, so Python never needs users.*

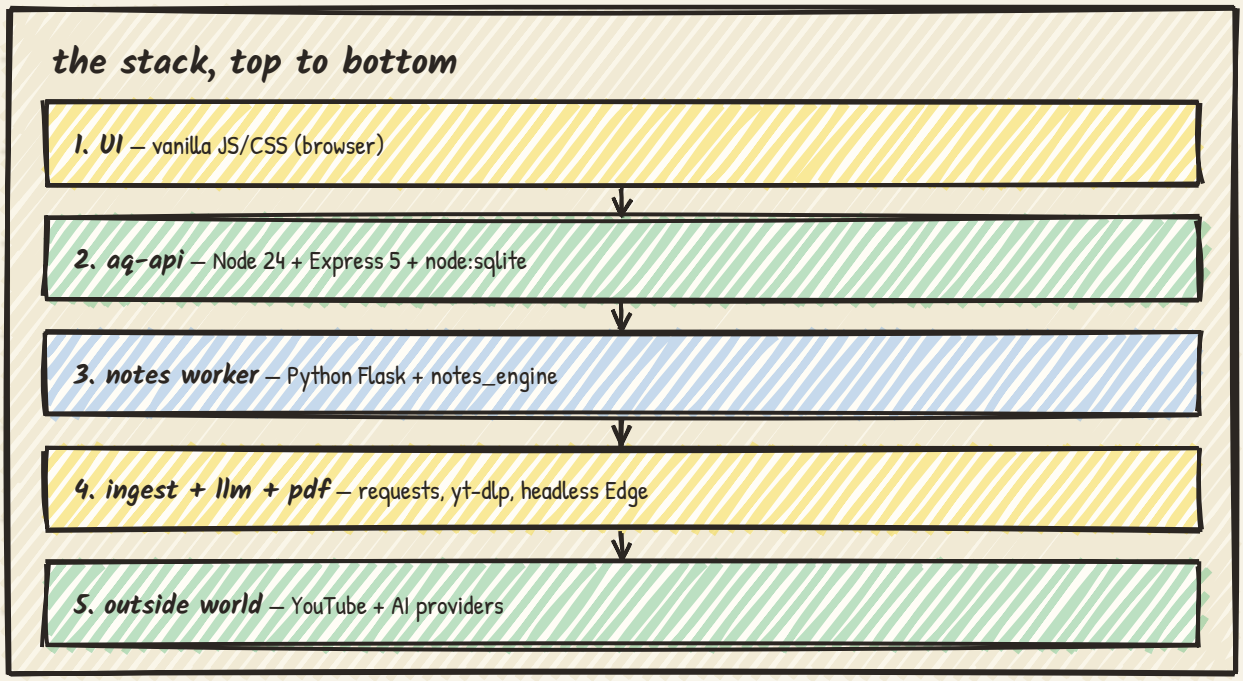
★ *recall question*

Q: Walk me through the architecture in one breath.

A: Browser UI -> aq-api (Express, auth middleware, SQLite for playlists/progress/notes records) -> for new notes it calls the Python worker, which fetches the transcript from YouTube, has an LLM write notes, validates them and renders HTML/PDF.

next → pg. 3: tech stack

3. tech stack — boring tools on purpose



each layer only talks to the one below

layer	tech	why this one
UI	vanilla JS + CSS, IIFE modules on `window.AQ`	no build, no npm; opens anywhere
App backend	Node >= 24, Express 5, built-in `node:sqlite`	one file DB, zero native deps
AI worker	Python 3.10+, Flask, flask-cors, python-dotenv	best ecosystem for transcripts + yt-dlp
Transcripts	youtube-transcript-api, yt-dlp, ffmpeg, Groq Whisper	a 3-step fallback chain
LLMs	Gemini, Groq, xAI Grok, Hugging Face via plain `requests`	no vendor SDKs, swap freely
Notebook look	Rough.js, bundled fonts (Kalam, Patrick Hand, Caveat)	sketchy, self-contained
PDF	headless Edge/Chrome `--print-to-pdf`, PyMuPDF for checks	HTML -> A4, page numbers verified
Tests	pytest (Python, all network mocked), `node --test` (Node)	no live calls needed

pro tip

Pattern: every dependency earns its place. The Python engine uses `requests` directly instead of five vendor SDKs, and the UI avoids a framework so it needs no toolchain.

★ to remember

→ *No framework on the UI, no SDKs in the worker, no native DB driver in Node.*

→ *Python is the only place that needs pip packages; Node needs just Express.*

→ *Edge or Chrome is only needed to print PDFs; HTML and JSON work without it.*

★ recall question

Q: Why plain `requests` instead of each provider's SDK?

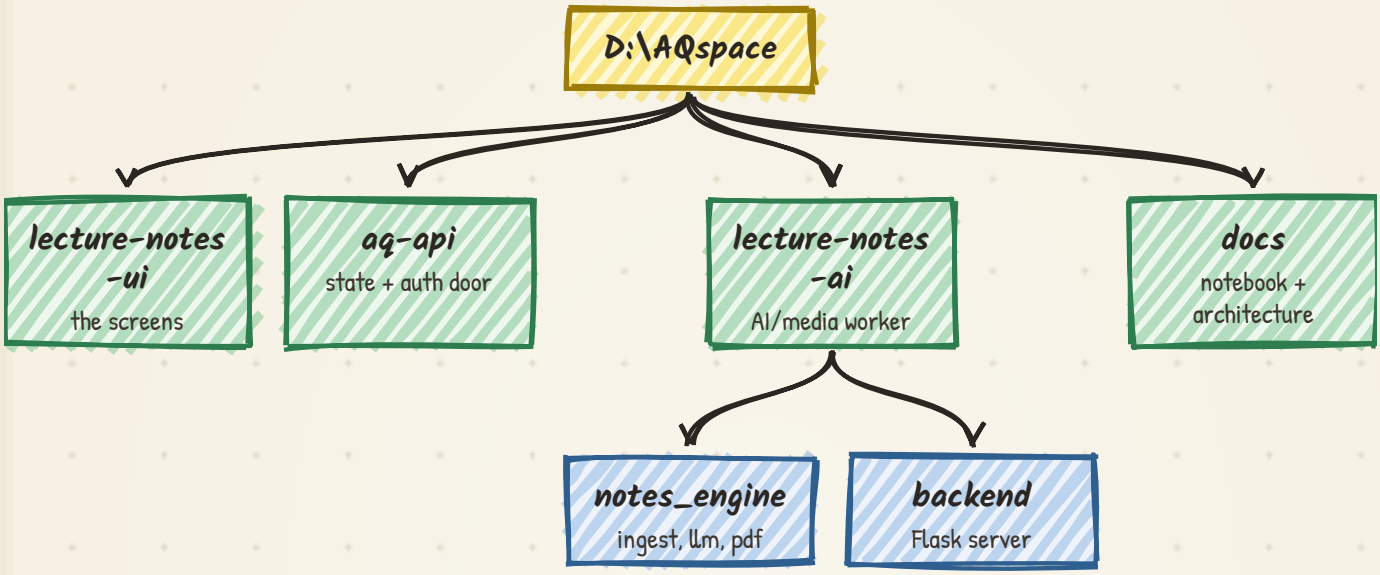
A: Four providers means four SDKs to install and keep compatible. One small `_Client` speaks the OpenAI-style or Gemini JSON over HTTP, handles retries and key scrubbing in one place, and adding a provider is a table entry.

next → pg. 4: repo layout

4. repo layout — one workspace, three apps + docs

Everything lives under `D:\AQspace`. The three apps are siblings with no shared code; they only share an HTTP contract. Two old folders are leftovers.

repo tree



what each folder is for

text

```
D:\AQspace
  lecture-notes-ui\   UI: index.html + 13 .js + styles.css (no build)
  aq-api\             Node app backend: src/ sql/ test/ data/aq.db
  lecture-notes-ai\   Python worker
     notes_engine\    ingest llm prompts notes_schema pdf
     backend\         Flask server.py + run.py + .env
     frontend\        old standalone page (still works)
     tests\ docs\     pytest suites, notes-schema.md
  docs\               this notebook + ARCHITECTURE.md
  backend\ frontend\  empty Vite/Express scaffolds (unused)
```

watch out!

`D:\AQspace\backend` (port 3005 hello-world) and `D:\AQspace\frontend` (React + Vite template) are unrelated scaffolds, not part of the running system.

★ ***to remember***

→ *lecture-notes-ui, aq-api and lecture-notes-ai are independent siblings.*

→ *notes_engine is a pip-installable package: CLI, library and web all use it.*

→ *Only an HTTP contract connects the apps, so each can be replaced alone.*

★ *recall question*

Q: Why keep the UI, API and worker in separate folders?

A: Different runtimes and different change rates: UI is static files, Node is stateful, Python is heavy and stateless. The seams are HTTP contracts, so any one can be swapped or deployed separately.

next → pg. 5: data layer

5. data layer — where each byte lives

State is split by ***who owns it***. Users' playlists and progress belong in the app database; generated files belong in the worker's job folders; the browser keeps only caches.

browser copy vs app database

localStorage
fast, per browser, offline-ok

SQLite via aq-api
shared, durable, per user

worker folders
generated files, 24h TTL

the UI-side cache and the Node source of truth

place	what	lifetime
SQLite `aq-api/data/aq.db`	users, videos, playlists, playlist_videos, watch_progress, notes_jobs	permanent
localStorage `aq.playlists.v1`	playlists + videos (UI-side copy / offline)	per browser
localStorage `aq.watch.v1`	{videoId: {pos, dur, done, updated}}	per browser
localStorage `aq.notes.v1`	{videoId: {jobId, state}} to resume polling	per browser
localStorage `aq.dirty.v1`	"1" = local edits not yet sent to aq-api	per browser
Python memory `State.jobs`	job status, progress, links	until restart / 24h
`backend/output/<job_id>/`	notes.json, notes.html, notes.pdf	deleted after 24h

note to self

The old standalone page also stores `lna.key.<provider>` and `lna.provider` (API key, only if you tick Remember). Keys are never written to disk server-side.

★ **to remember**

→ *Playlists and progress are durable data: they live in SQLite behind Node.*

→ *Python holds only temporary job state and files; a restart loses job ids.*

→ *The notes record cached by video_id means two users never pay twice for one video.*

★ *recall question*

Q: Why cache notes by video_id in Node when Python already writes files?

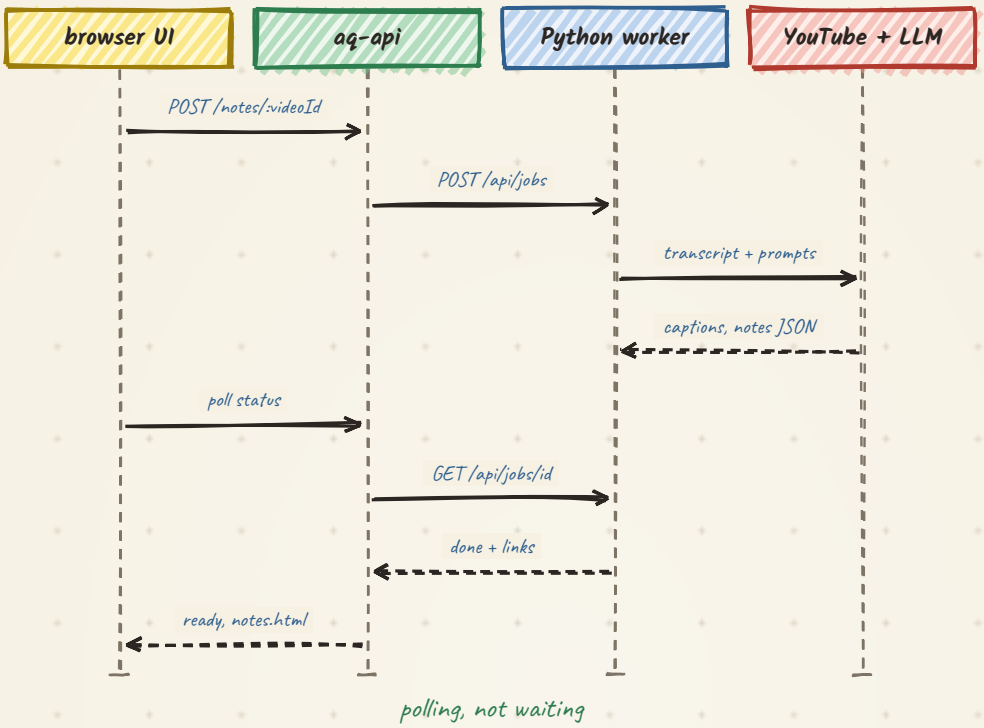
A: Python's files are temporary (24h) and its job ids die on restart. A Node record keyed by the 11-char video id survives, so a video is generated once and every playlist, user and device reuses it.

next → pg. 6: request flow: get notes

6. request flow: get notes — click to PDF, end to end

The slowest path in the system. The UI never waits on one long request: it starts a job and **polls**.

notes generation



1. UI sends `POST /api/v1/notes/<videoId>` to aq-api (video must be in one of the user's playlists).
2. Node checks `notes_jobs` for a live record of that video_id; done or running is returned at once (200), nothing new starts.
3. On a miss Node calls Python `POST /api/jobs` {url, provider}, gets a job_id and inserts a queued record (202).
4. Python thread: transcript (0–25%), LLM notes (25–85%), HTML+PDF (85–100%).
5. UI polls `GET /api/v1/notes/<videoId>` about every 1.5 s; Node refreshes from Python `GET /api/jobs/<id>` at most every 0.8 s.
6. On done Node marks the record done; the UI frames `/notes/<videoId>/html`, which Node streams from Python.

pro tip

Max 2 jobs run at once on the server; the UI queues extra videos client-side. When `AQ.api.available()` is false the UI falls back to calling Python on :5055 directly.

★ ***to remember***

→ *The UI polls; it never holds a request open for minutes.*

→ *Progress is stage-mapped: transcript 0–25, notes 25–85, render 85–100.*

→ *A cache hit short-circuits everything after step 2.*

★ *recall question*

Q: Why a job + polling instead of one long HTTP request?

A: Notes take seconds to minutes (long lectures need many LLM calls). Proxies and browsers time out long requests; a job returns 202 immediately and exposes stage and percent for a progress bar.

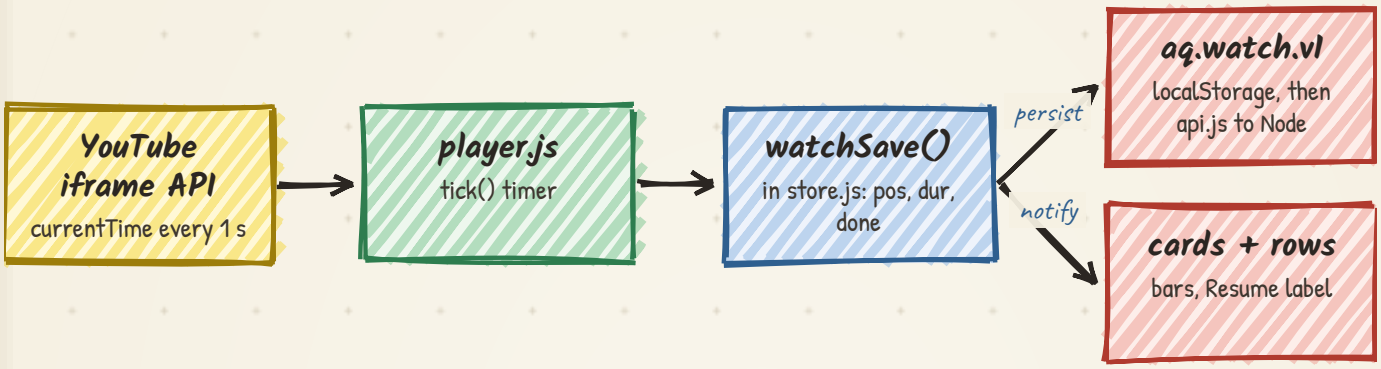
next → pg. 7: watch & resume flow

7. watch & resume flow

— the player remembers where you stopped

`player.js` embeds a `youtube-nocookie.com` iframe and loads the YouTube IFrame API once. A 1-second timer reads the ***real*** playback position and hands it to the store.

watch state loop



from the YouTube iframe to the progress bars

1. Open a video: `AQ.store.watchInfo(id)` gives pos/dur; start > 0 shows "Resumed at m:ss".
2. Every second `tick()` calls `S.watchSave(id, {pos, dur})`; storage writes are silent.
3. At >= 95% of the duration, or on the ended event, `done` becomes true.
4. Replaying a finished video and stopping early flips it back to Resume.
5. On dialog close `S.notify()` re-renders cards and rows (bars, labels); `api.js` sends the record to aq-api (debounced 2 s).
6. `learnDuration()` fills a missing duration from the real player.

note to self

Rows get their button text from `watchLabel`: Play, Resume * 12:34, or Replay. The `storage` event keeps other tabs in sync.

> ★ **to remember**
>
> → *The position is measured, not guessed: it comes from the player itself.*
>
> → *done = ended event or >= 95% of duration.*
>
> → *Memory is the source of truth; a blocked localStorage never breaks playback.*

★ *recall question*

Q: How does resume work, and what if the YouTube API is blocked?

A: A 1 s timer stores {pos, dur} per video id; reopening starts at pos. If the IFrame API cannot load, the plain embed still plays, just without saved progress.

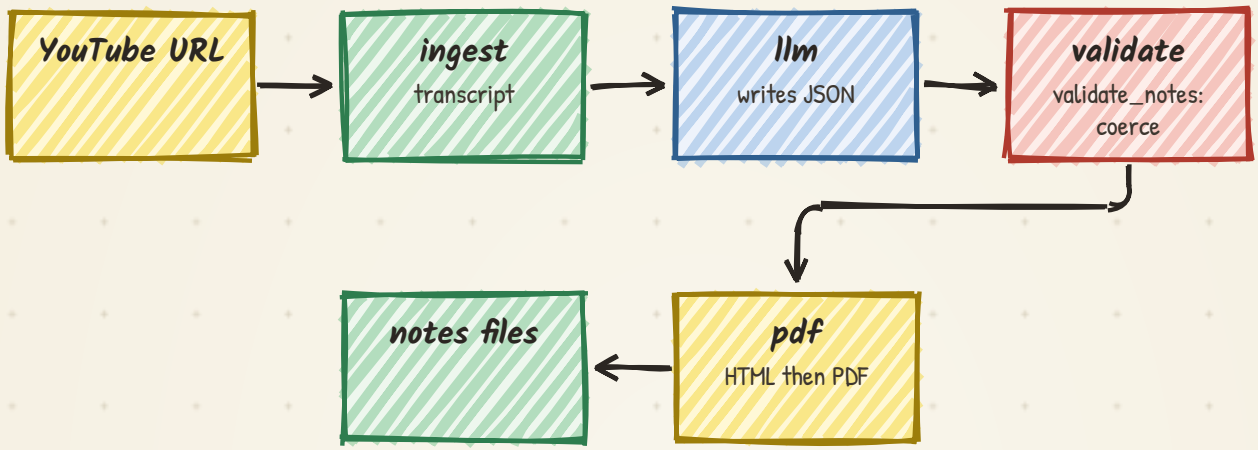
next → pg. 8: notes pipeline in Python

8. notes pipeline in Python

— URL -> transcript -> notes -> notebook

`notes_engine.generate(url, provider, ...)` is the whole worker in one call; the Flask server runs the same stages with extra status reporting. Progress is remapped to one global 0–100 bar.

engine stages



each stage can fail with its own user-safe error

stage	module	output	pct
ingest	`ingest.get_transcript`	title, channel, duration, timed segments	0–25
write	`llm.generate_notes`	raw JSON from the model	25–85
clean	`notes_schema.validate_notes`	contract-safe Notes dict	~97
draw	`pdf.render_html / render_pdf`	notes.html, notes.pdf	85–100

The same engine has three front doors: the Flask web server, the CLI `python -m notes_engine URL` and the library import. Exit codes tell a script what failed: 3 transcript, 4 AI provider, 5 no PDF.

> ★ **to remember**
> → *generate() = ingest -> llm -> validate -> render, with one global progress bar.*
> → *IngestError, LLMError and PdfError carry messages that are safe to show users.*
> → *If only the PDF fails, HTML and JSON are still saved and a warning is returned.*

★ *recall question*

Q: What happens if the PDF step fails but the notes were written?

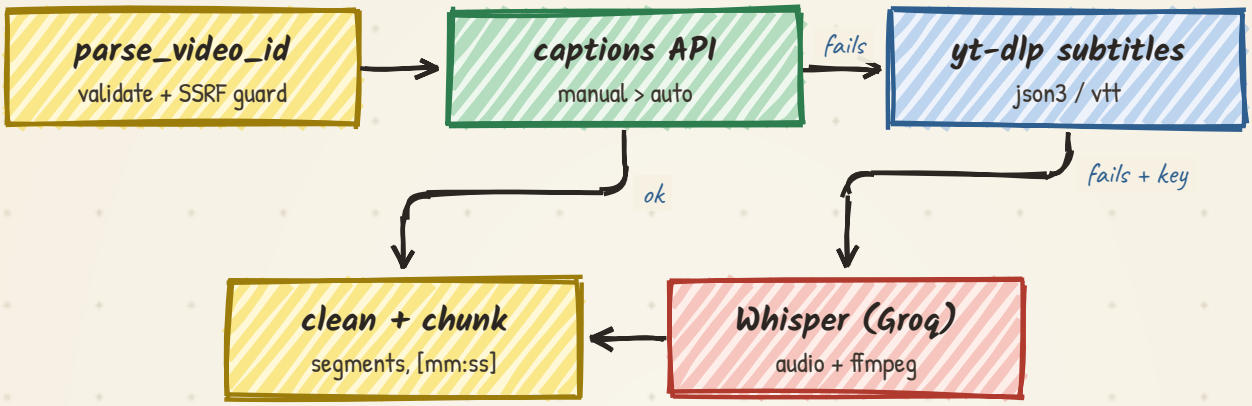
A: The job still finishes as done with a warning: notes.json and notes.html are kept and linked, pdf_path is None. Only when both HTML and PDF fail does the job become an error.

next → pg. 9: ingest.py: transcripts

9. ingest.py: transcripts — three fallbacks, one clean format

`ingest.py` (1,200 lines) turns a link into `{video_id, title, channel, duration_sec, language, source, segments[]}`. It validates the link first, which doubles as the SSRF guard: only 11-char ids from known YouTube hosts get through.

fallback chain



stop at the first method that works

1. `parse_video_id` accepts watch, youtu.be, shorts, embed, live; anything else is rejected.
2. Metadata: yt-dlp info, falling back to the oEmbed endpoint.
3. Try 1: youtube-transcript-api, manual captions before auto, English first.
4. Try 2: yt-dlp subtitle download (json3 or vtt).
5. Try 3 (only with a Groq key): download audio, ffmpeg 16 kHz mono in 10-minute chunks, Groq Whisper.
6. Clean: strip sound cues, dedupe overlapping cues, merge into sentences.

- `chunk_transcript` slices text for the LLM (12,000 chars, 400 overlap) with a `[mm:ss]` marker every 30 s.
- `get_playlist` lists up to 200 videos via yt-dlp, skipping private/deleted ones.

> ★ **to remember**
> → Order: captions API, yt-dlp subtitles, then Whisper only if a Groq key exists.
> → Private, live and unavailable videos stop the chain early with a clear message.
> → Cleaning and 30 s time markers let the LLM stamp sections with timestamps.

★ *recall question*

Q: Why validate the URL so strictly before any network call?

A: The server fetches whatever it is given, so an unchecked URL could point it at internal addresses (SSRF). Only an 11-char id from allowed YouTube hosts passes, and the player URL is rebuilt from that id.

next → pg. 10: llm.py & prompts.py

10. llm.py & prompts.py — *single pass vs map-reduce*

`llm.generate_notes` picks a strategy from transcript size and the provider's limits (`_profile`). `prompts.py` holds the rules (knowledge only, faithful, 2-4 takeaways, one recall question per section) and four message builders.

two strategies

single pass
1 call -> whole Notes JSON

map-reduce
digest slices, compose, finalize

short lectures get one call, long ones are map-reduced

provider	default model	note
gemini	gemini-2.5-flash	huge context, 32k output
groq	openai/gpt-oss-120b	free tier ~8k tokens/min, paced client-side
huggingface	Llama-3.3-70B-Instruct	needs Inference Providers token
xai	grok-4	paid API

- Target size: about one section per 2.5 minutes, clamped to 6-18.
- A `_Pacer` keeps the trailing minute under ~85% of the token limit; 429s sleep for Retry-After.
- Up to 5 attempts per call; bad JSON is repaired (`fix_json_messages`) or salvaged if truncated.
- The key only goes in a header and is scrubbed from every error message.

> ★ **to remember**
> → *Short lecture: one call. Long lecture: digest each slice, compose batches, write cover/glossary/recap.*
> → *Single pass that returns too few sections falls back to map-reduce.*
> → *Rate limits are handled by pacing before sending, not only retrying after 429.*

★ *recall question*

Q: How do you handle a lecture longer than the model's context or rate limit?

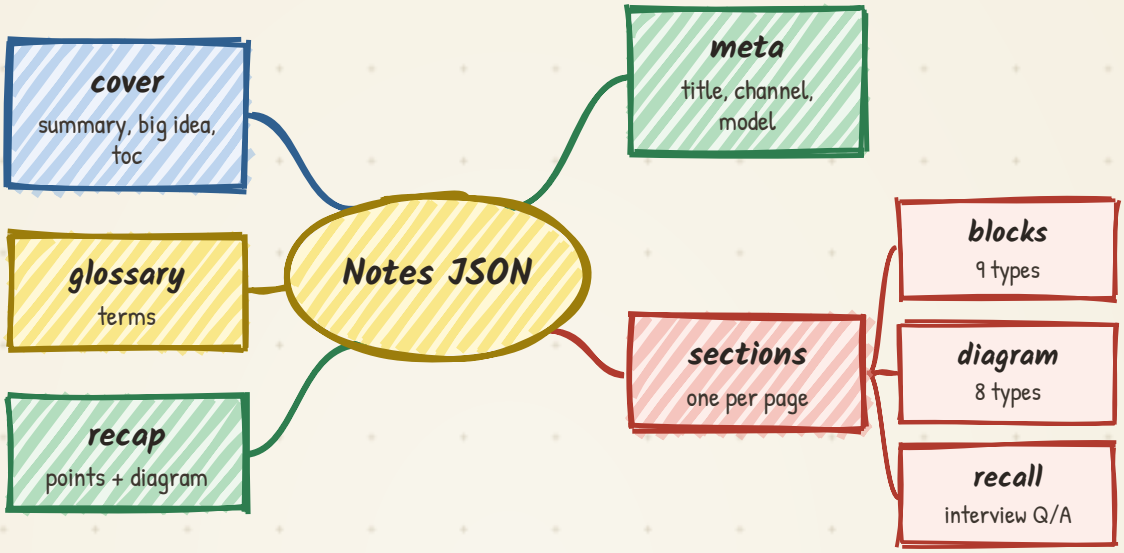
A: Map-reduce: chunk the transcript, make a compact digest per chunk, pack digests into batches that each compose 2-10 sections, then one small call writes subtitle, glossary and recap. A pacer and retries keep each call inside the provider's token-per-minute budget.

next → pg. 11: notes_schema.py: contract

11. notes_schema.py: contract — *forgiving in, strict out*

LLM output is messy, so `validate_notes` ***coerces instead of rejecting***. It raises only when no section survives. Its output is the single contract between prompts, renderer and UI (`docs/notes-schema.md`).

the Notes JSON



everything the renderer needs

- Section numbers are always rebuilt as 2, 3, 4... (page 1 is the cover).
- `cover.toc` and every `next` footer are regenerated from the final list, so they cannot drift.
- Facts (title, channel, duration, url, language) come from the transcript, not the model.
- Caps: 24 sections, 14 blocks, 8 diagram nodes (mindmap 10), labels 40 chars, 30 glossary terms.
- Aliases are mapped: `flowchart` -> flow, `mind map` -> mindmap, `children` lists -> `parent` ids.

pro tip

`validate_notes(validate_notes(x))` is stable: running it twice changes nothing. That is what makes it safe to re-validate stored notes.

★ ***to remember***

→ *validate_notes coerces: it only fails when there is nothing usable.*

→ *Derived fields (toc, next, numbers) are rebuilt, never trusted from the model.*

→ *9 block types and 8 diagram types are the whole vocabulary.*

★ *recall question*

Q: Why coerce LLM JSON instead of rejecting bad output?

A: Retrying costs tokens and time, and models are nearly right most of the time. Clamping lengths, fixing aliases and rebuilding derived fields rescues almost every answer deterministically, with no extra API calls.

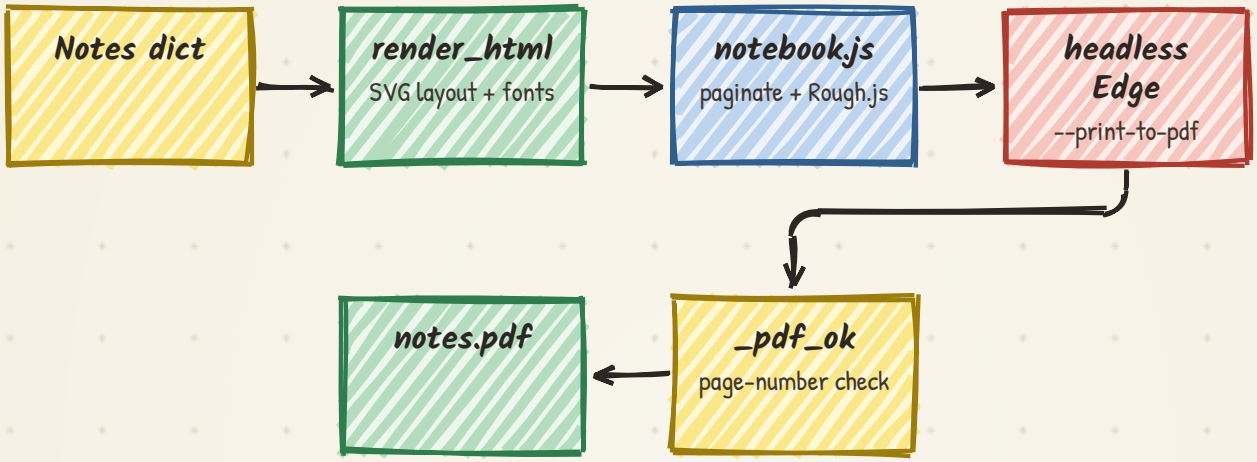
next → pg. 12: pdf.py: the notebook

12. pdf.py: the notebook

— Python lays out, the browser paginates

`render_html` builds one self-contained page: fonts inlined as base64, SVG diagrams laid out ***in Python*** (text measured with the bundled fonts via PyMuPDF so labels never overflow), then the browser script paginates and sketches.

HTML to PDF



layout in Python, pagination in the browser

1. Escape all text (`html.escape`); re-allow only ***bold*** and `code`.
2. Diagram builders (`_svg_flow`, `_svg_stack`, ..., `_svg_sequence`) emit SVG with plain strokes + seeded shape specs.
3. `templates/notebook.js` pours units into fixed A4 page boxes, splitting lists, tables and code across pages.
4. Vendored `rough.js` redraws the shapes hand-drawn; every page gets rings, footer and `pg. N / M`.
5. `render_pdf` runs headless Edge/Chrome `--print-to-pdf` (retry with a longer virtual-time budget).
6. `_pdf_ok` reads the PDF back: every page must say `pg. N / M` with M = real page count.

> ★ **to remember**
>
> → *Text is escaped before any markup is added back: notes cannot inject HTML.*
>
> → *Diagrams are measured in Python so nothing overflows; Rough.js only adds the wobble.*
>
> → *A PDF is accepted only when every page's `pg. N / M` matches the page count.*

★ *recall question*

Q: How do you know the generated PDF is correct?

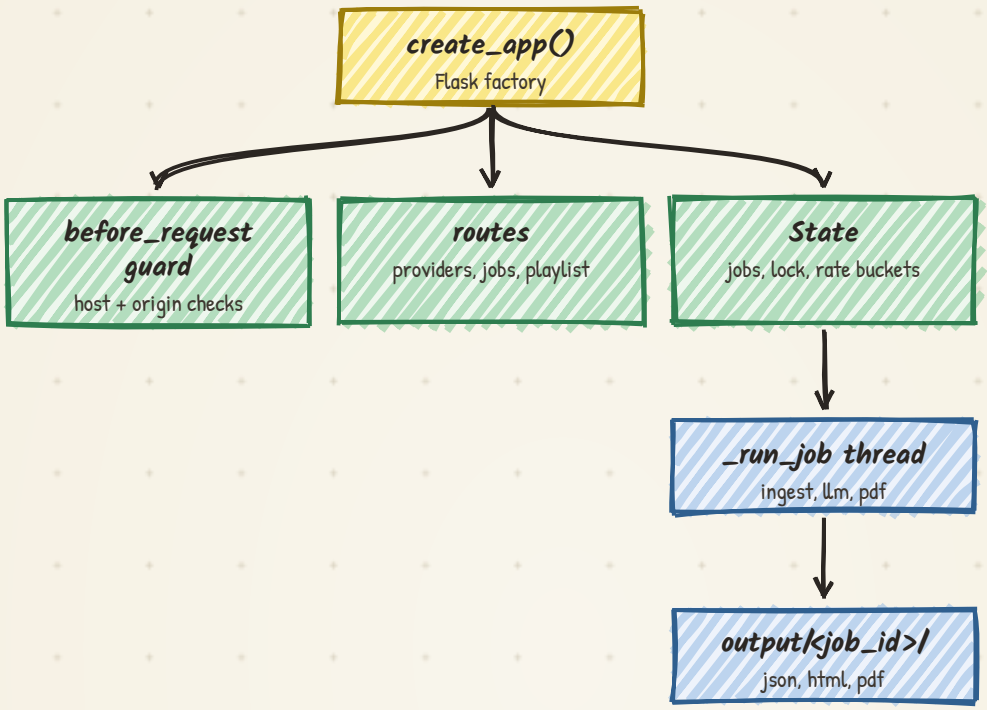
A: After printing, PyMuPDF reads every page's footer; the PDF is accepted only if each page says pg. N / M with M equal to the real page count. Otherwise it retries with more render time, then raises PdfError.

next → pg. 13: Flask server: the worker

13. Flask server: the worker — jobs in memory, files on disk

`backend/app/server.py` (650 lines) is an app factory, `create_app()`. Modules `ingest`, `llm`, `pdf` are imported ***lazily inside the worker*** so the server still starts if one is broken.

server anatomy



request guard, then routes, then a job thread

method + path	***does***
GET `/api/providers`	providers, default models, `server_key` yes/no
POST `/api/jobs`	validate {url, provider, api_key?, model?, groq_key?} -> 202 {job_id}
GET `/api/jobs/<id>`	status, stage, pct, message, meta, links, warning, error
GET `/api/jobs/<id>/notes.json|pdf|html`	the three outputs (html allowed in a local iframe)
GET `/api/playlist?url=`	playlist metadata via yt-dlp (max 200 videos)

- `State` holds the jobs dict, a lock, a 10-per-60 s rate bucket, a separate playlist bucket and 2 playlist slots.
- Each job runs in a daemon thread `_run_job`; max 2 active, otherwise 429.
- `cleanup_old_jobs` deletes job folders older than 24 h at startup.

> ★ ***to remember***
> → *Lazy imports keep the server alive even with a broken module.*
> → *Concurrency is bounded: 2 active jobs, 10 new jobs per 60 s, 2 playlist fetches.*
> → *Job status is in memory (restart = 404); files on disk last 24 hours.*

★ *recall question*

Q: Why run jobs in threads with in-memory state instead of a queue?

A: It is a single-user local worker, so a thread plus a dict is the simplest thing that works. The cost: a restart loses job ids, which is why Node caches the finished notes record by video_id.

next → pg. 14: python files, one line each

14. python files, one line each

— every file in lecture-notes-ai

file	role
`notes_engine/__init__.py`	`generate()`, `render_notes()`, lazy re-exports, ENV_KEYS
`notes_engine/__main__.py`	CLI, progress bar, exit codes 0–5
`notes_engine/ingest.py`	link validation, transcripts, playlists, chunking
`notes_engine/llm.py`	providers, pacing, single pass / map-reduce
`notes_engine/prompts.py`	rules + message builders (digest, compose, final, fix-json)
`notes_engine/notes_schema.py`	`validate_notes` coercion + limits
`notes_engine/pdf.py`	diagram SVG layout, HTML build, headless PDF
`templates/notebook.js` / `.css`	pagination, rings, footer / paper + print styles
`templates/vendor/rough.js`	Rough.js sketchy shapes (MIT)
`static/fonts/*.ttf`	Kalam, Patrick Hand, Caveat, JetBrains Mono (OFL)

file	role
`backend/app/server.py`	Flask app factory, jobs, guards
`backend/run.py`	starts Flask on 127.0.0.1: `LNA_PORT` (5055)
`backend/.env.example`	template for provider keys
`frontend/*`	old standalone page (form, stepper, preview)
`start.ps1` / `pyproject.toml`	one-click launcher / package + extras
`tests/`, `docs/`	pytest suites + fixtures / notes-schema.md

★ to remember

→ *notes_engine has no web code; the Flask server is a thin shell around it.*

→ *The CLI, the library and the server all call the same generate stages.*

→ *Fonts, templates and Rough.js ship inside the package, so PDFs are reproducible.*

★ recall question

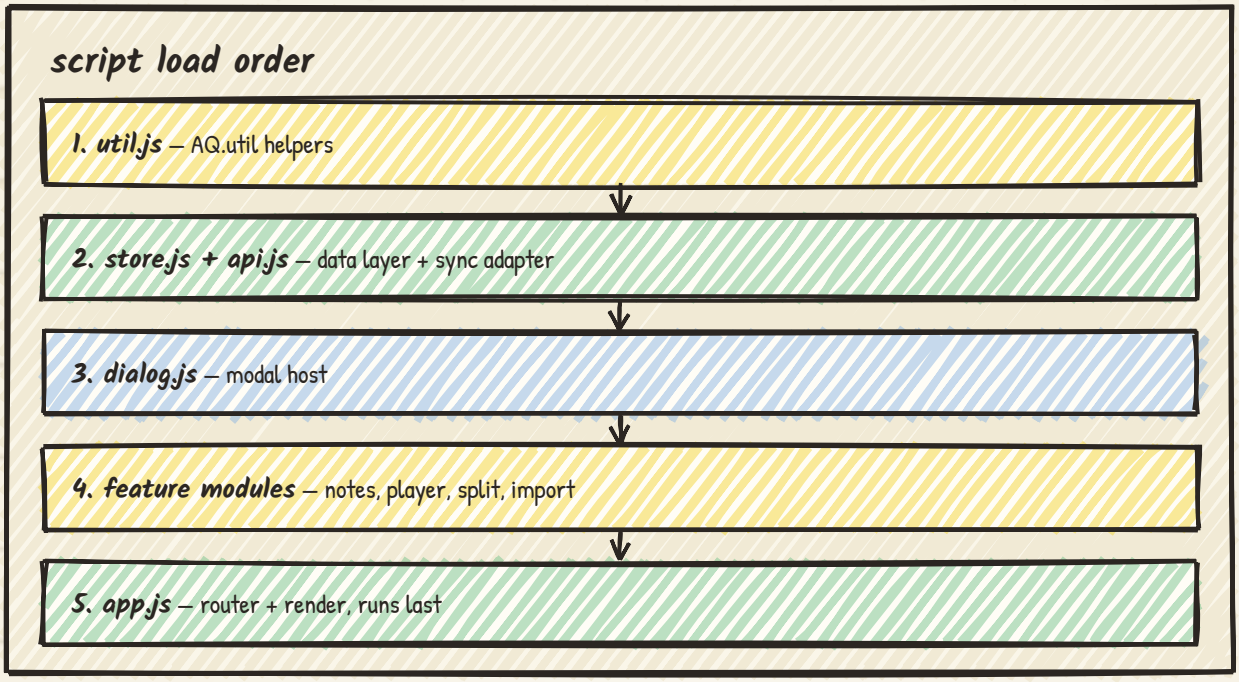
Q: Where would you add a fifth LLM provider?

A: In `llm.PROVIDERS` (label, default model, key hint) and `_ENDPOINTS`, a size profile in `_BASE_PROFILES`, and the env var in `ENV_KEYS`; the server and UI read the provider table, so they pick it up automatically.

next → pg. 15: UI core files

15. UI core files — shell, data, helpers, screens

No bundler: `index.html` loads plain scripts in dependency order, and every file is an IIFE that hangs its API on one global, `window.AQ`.



later files depend on earlier ones

file	role	talks to
`index.html`	shell: top bar, `#view`, modal, script order	loads all .js
`styles.css`	tokens + all components, black glass theme	-
`util.js`	`el()`, icons, YouTube id parser, durations	used by all
`store.js`	playlist CRUD, sanitising, watch progress, limits	localStorage, `api.js`
`api.js`	sync adapter: probes aq-api, imports once, writes through, dirty flag	aq-api, store
`dialog.js`	accessible modal: focus trap, Esc, restore focus	`#modal`
`app.js`	hash router, home grid, chroma cards, detail, drag-and-drop	store, dialog, split

note to self

Safety rule across the UI: user text is only set with `textContent`; there is no `innerHTML` anywhere.

★ **to remember**

→ *One global namespace (AQ) and ordered script tags replace a bundler.*

→ *store.js has no DOM code; app.js renders from store and re-renders on notify().*

→ *Routes: `#/` home, `#/playlist/<id>`, `#/playlist/<id>/notes/<videoId>`.*

★ *recall question*

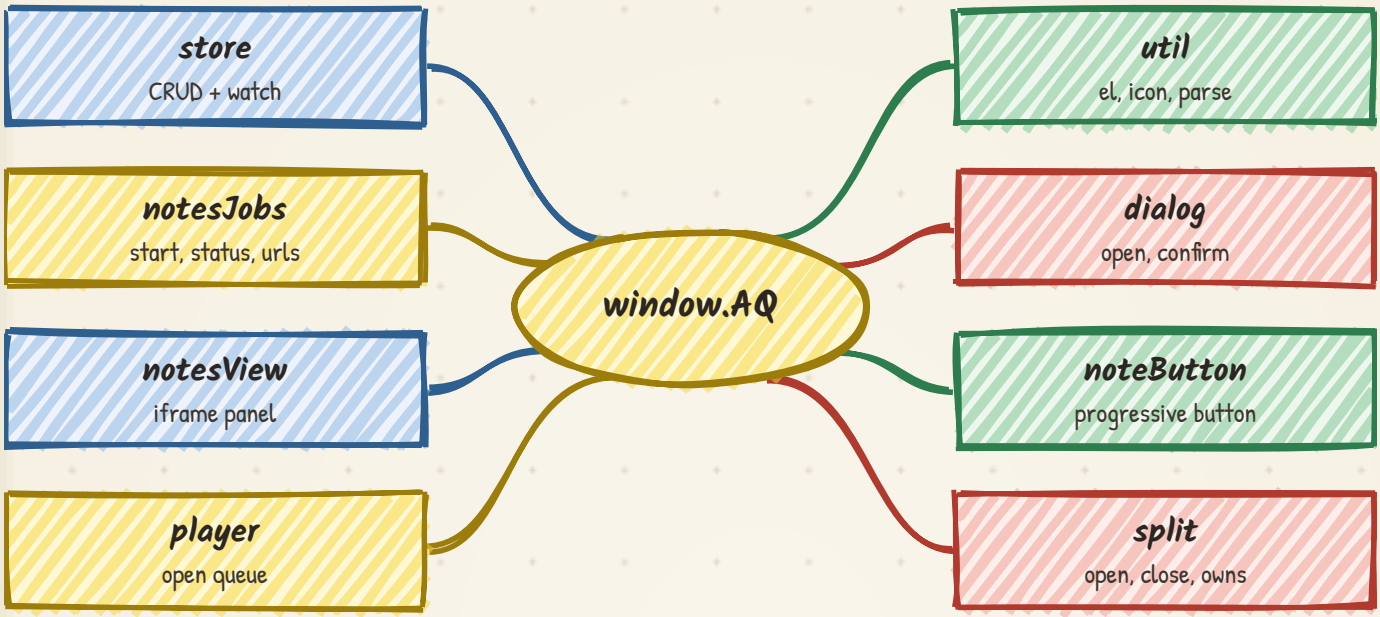
Q: How is the UI organised without a framework or build step?

A: Small IIFE modules sharing one `window.AQ` namespace, loaded in order. `store.js` owns data and emits change events, `app.js` listens, re-renders the active hash route and writes the DOM only through textContent.

next → pg. 16: UI feature modules

16. UI feature modules — player, notes, import, effects

window.AQ namespace



who exposes what

file	role	talks to
`player.js`	youtube-nocookie embed, prev/next, saves position every 1 s	YouTube IFrame API, store
`notes.js`	`AQ.notesJobs`: start/poll/retry/urls, max 2 running, resumes after reload	aq-api `/notes`, else :5055; `aq.notes.v1`
`notebutton.js`	Get notes button: none, running %, ready, retry	notesJobs
`notesview.js`	notes panel: iframe of notes.html, Download PDF	notesJobs, notebutton
`split.js`	slide-in notes panel, list shrinks to nav (FLIP animation)	notesView, history
`importpl.js`	import dialog: fetch playlist, preview, create local copy	aq-api `/import/youtube`, store
`water.js`	liquid-glass New playlist button (SVG refraction)	-
`clickspark.js`	click spark canvas effect	-

★ **to remember**

→ *notesJobs is the shared brain: rows, player and button all read it.*

→ *split.js keeps the SAME list node across states, so morphing is a cheap FLIP animation.*

→ *All motion respects prefers-reduced-motion.*

★ *recall question*

Q: Why does notes.js keep job ids in localStorage?

A: Server job ids are in memory and the page may reload mid-generation. Storing {videoId: {jobId, state}} lets queued or running jobs resume polling, and a 404 from the server turns into "Notes expired, generate again".

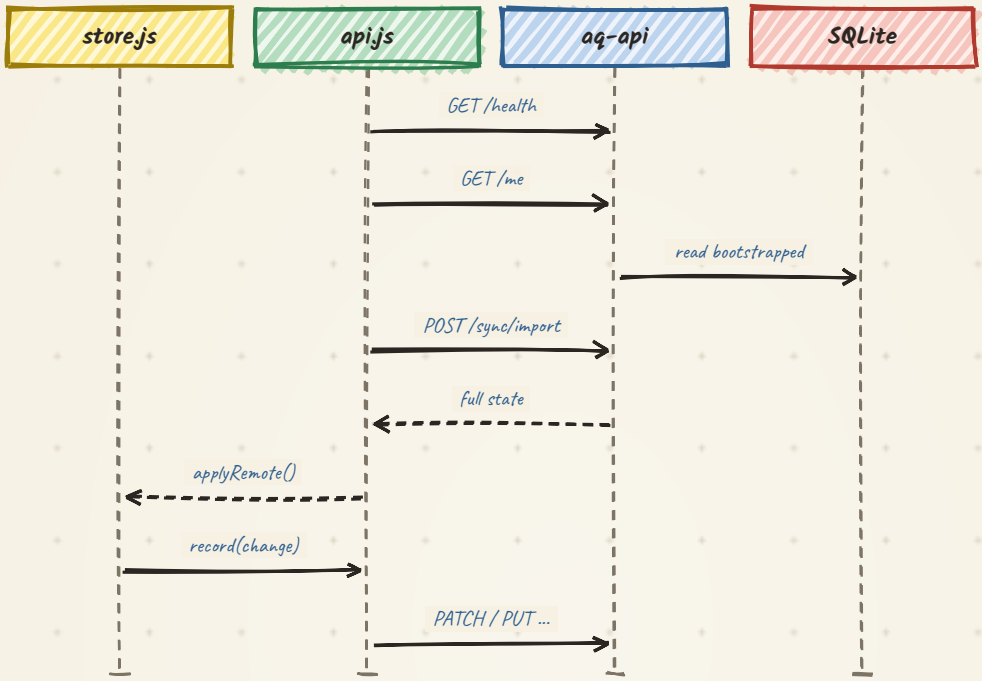
next → pg. 17: api.js: the sync adapter

17. api.js: the sync adapter

— local first, server as source of truth

`store.js` stays ***synchronous*** and keeps `localStorage` as the offline cache. `api.js` (loaded right after it) mirrors everything to aq-api, so the UI never waits on the network and still works if the server is down.

sync on start



probe, reconcile, then write-through

1. Boot: probe `GET /api/v1/health` on the same origin, then on `http://127.0.0.1:5180`.
2. `reconcile()`: `GET /me`. If not `bootstrapped` (or the dirty flag is set) push local data with `POST /sync/import`; otherwise pull playlists + progress.
3. `store.applyRemote()` adopts the server's data (a newer local watch record still wins) and re-renders.
4. Every store change calls `AQ.api.record(name, args)`; writes are queued ***in order*** (create, PATCH, DELETE, PUT order...).
5. Watch progress is debounced: one `PUT /progress/<videoId>` per video every 2 s, flushed on tab hide / `pagehide`.
6. Server unreachable: `aq.dirty.v1 = 1`; on `online` or tab focus the whole local state is pushed again.

note to self

Last writer wins by the `updated` timestamp: the server ignores an older progress write than the one it holds.

★ ***to remember***

→ *localStorage is the cache, aq-api is the source of truth; the UI code stays synchronous.*

→ *Writes go through in order; progress is debounced to one PUT per video per 2 s.*

→ *A dirty flag guarantees offline edits are pushed later, never lost.*

★ *recall question*

Q: What happens to edits you make while the server is down?

A: They are saved to localStorage immediately and `aq.dirty.v1` is set. When the page regains focus or the browser goes online, api.js reconnects and pushes the whole local state with POST /sync/import in replace mode, then clears the flag.

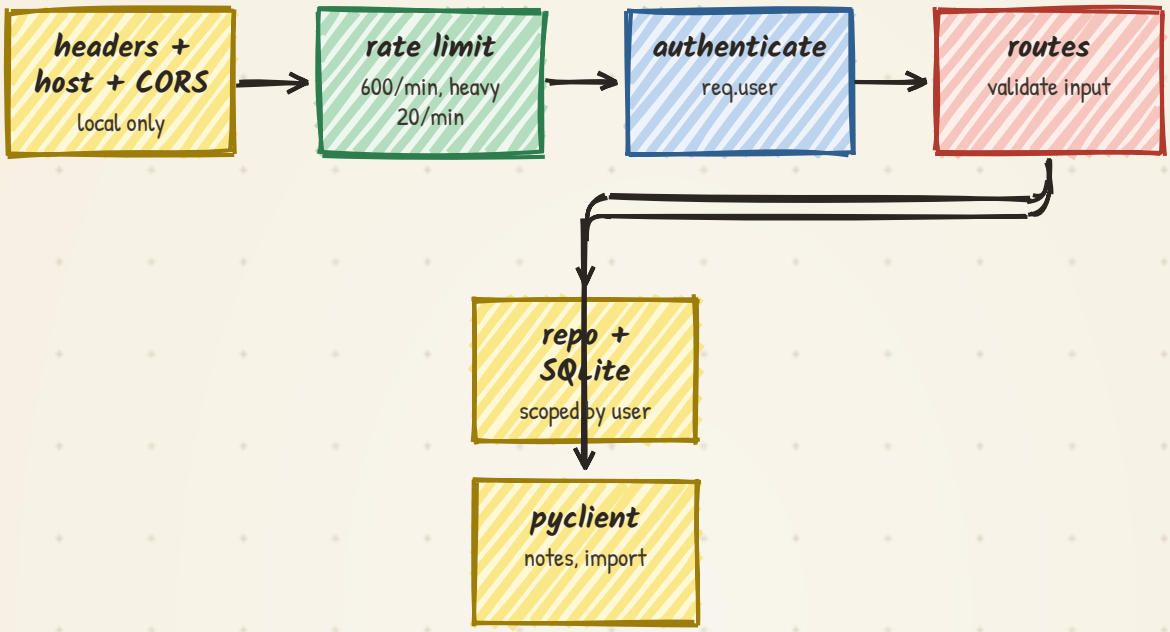
next → pg. 18: aq-api: the Node backend

18. aq-api: the Node backend

— state, identity and the front door

`aq-api` owns users, playlists, videos, watch progress and one notes record per video_id. It proxies the two slow things (notes, YouTube import) to Python and serves the UI on :5180. One dependency: `express`.

a request inside aq-api



guards first, then auth, then the route

file	role
`src/server.js`, `config.js`	entry + env config (port, DB, Python URL, limits)
`src/app.js`	`createApp`: middleware order, routers, static UI, JSON errors
`src/db.js`, `repo.js`	node:sqlite + migrations; all SQL scoped by userId
`src/notes-service.js`	notes records + cache by video_id
`src/pyclient.js`	fixed-path client for the Python worker
`middleware/auth.js`, `security.js`	auth seam (local user today); headers, host guard, CORS, rate limit
`routes/*.js`, `sql/001_init.sql`	playlists, progress, notes, misc; 6 tables

- Routes live under `/api/v1/...`: playlists, progress, notes, import/youtube, sync/import, health, me.
- `authenticate` sets `req.user` before every data route; swap it for OAuth via `createApp(config, {authenticate})`.
- Node >= 24: SQLite is the built-in `node:sqlite`, no native module to compile.

> ★ ***to remember***
> *→ Node owns state and identity; Python stays stateless.*
> *→ Auth is a middleware seam: replace localAuth with an OAuth verifier, nothing else changes.*
> *→ The notes_jobs table is the cache: one generation serves every user and playlist.*

★ recall question

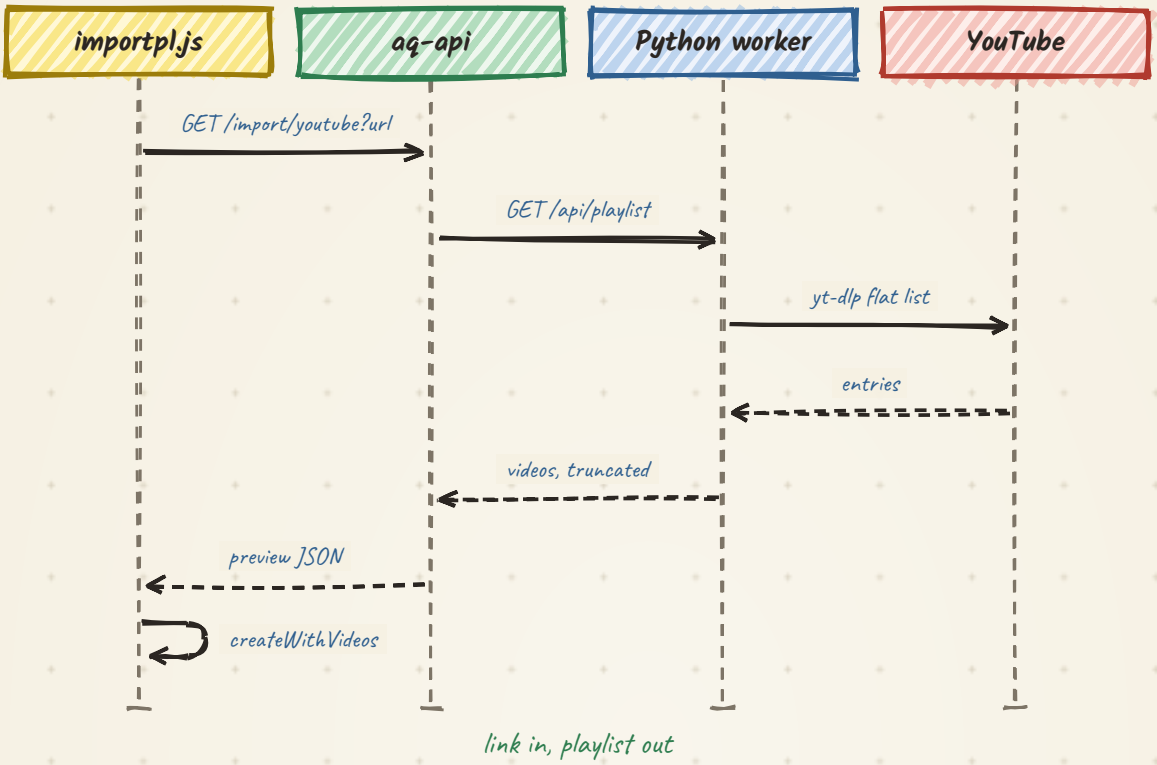
Q: Why add a Node backend instead of extending the Python server?

A: Python is a heavy, slow-job worker with in-memory jobs. Users, playlists and progress need a durable DB and an auth seam, which Express + SQLite does simply, and Node gives one front door that proxies the AI work.

next → pg. 19: playlist import flow

19. playlist import flow — paste a link, get a local copy

import



1. `importpl.js` extracts the `list=` id (10–64 chars) and asks aq-api: `GET /api/v1/import/youtube?url=`.
2. Node re-checks the id and forwards only a canonical playlist URL; Python validates again, then yt-dlp lists entries.
3. Server returns {id, title, channel, count, videos[], truncated, skipped}; private/deleted videos are skipped.
4. `cleanResponse` re-validates every field; the dialog previews title, count, total time, thumbnails.
5. `uniqueName` suggests "Title (2)" when the name is taken.
6. `store.createWithVideos(name, desc, videos, source)` writes once, with `source: {type: youtube, listId, url, importedAt}`.

note to self

Nothing syncs back to YouTube. The imported playlist is a copy; you can still add your own videos.

★ **to remember**

→ Validation happens twice: in the browser for UX, in Python for safety.

→ Limits: 200 videos per import, 2 playlist fetches at once, own rate bucket.

→ The response is treated as untrusted and re-sanitised by cleanResponse.

★ *recall question*

Q: Why re-validate the playlist response in the UI when the server already did?

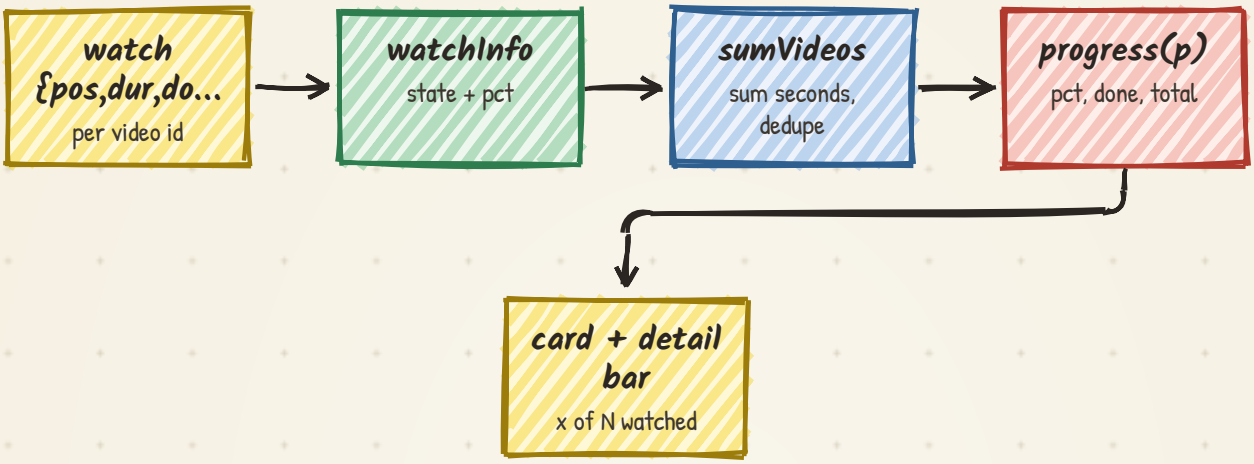
A: Defence in depth: the UI never trusts network data. cleanResponse drops bad ids, trims long text and caps the list, so a compromised or buggy server cannot inject markup or oversized data.

next → pg. 20: progress calculation

20. progress calculation — *time-based, not count-based*

Progress is ***watched seconds divided by total seconds***, so one 3-hour lecture weighs more than ten 2-minute clips. It is computed on the fly by `sumVideos` in `store.js`; nothing stores a percentage.

progress pipeline



from player seconds to the bar on a card

pct = round(100 * watchedSec / totalSec) (done video counts its full duration)

- Each unique video is counted once, even if it sits in several playlists.
- Per video: `watchInfo` -> state new / partial / done; done = ended or >= 95%.
- Cap rules: never 100% while seconds are missing; never 0% once something was watched.
- No known durations at all: fall back to finished videos / total videos.
- `progressTime` renders "14 of 36 min" or "1 hr 2 min of 4 hr 30 min+" (+ means some durations unknown).

e.g. worked example

Playlist of 3 videos: 10:00 (done), 20:00 (watched 5:00), 30:00 (new). watched = 600 + 300 = 900 s, total = 3600 s, pct = 25%, done = 1 of 3.

★ to remember

→ Percent is derived each time from seconds; it is never stored.

→ Duplicates across playlists are counted once, in `overall()` too.

→ Edge rules prevent a misleading 0% or 100%.

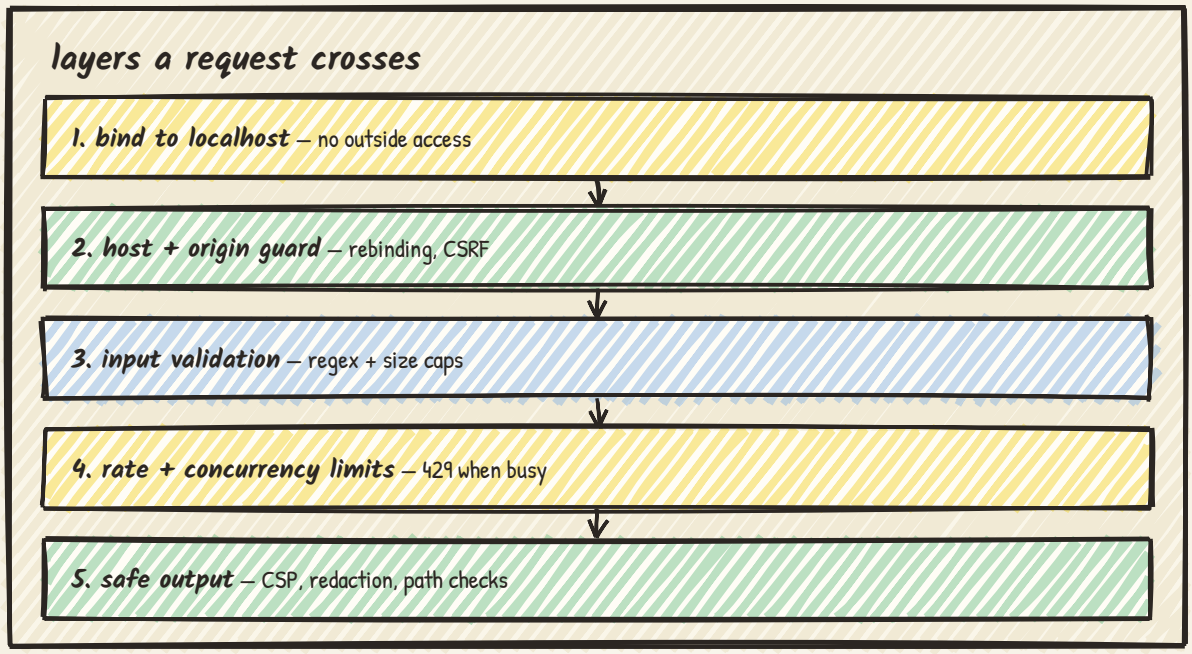
★ recall question

Q: Why weight progress by time instead of counting finished videos?

A: Videos differ wildly in length; counting them makes a 2-minute clip equal to a 3-hour lecture. Seconds watched over total seconds matches what the user feels, with a video-count fallback when durations are unknown.

next → pg. 21: security

21. security — defence in layers



outermost first

layer	control
Network	Python and Node both bind 127.0.0.1 only; never 0.0.0.0
Host / origin	DNS-rebinding guard: only localhost names; POST and `/api/playlist` need a local Origin
CORS	only localhost / 127.0.0.1 origins, GET/POST, `Content-Type` header
Input	key regex, model regex, 2048-char URL cap, 64 KB body cap, 11-char video ids
Secrets	API keys live in memory for one job; `_redact` scrubs logs and errors; never returned by the API
Headers	CSP per route, nosniff, frame options, no-referrer, same-origin resource policy
Notes preview	notes.html allowed only its own inline scripts by SHA-256 hash; framed only by local pages
UI	textContent only (no innerHTML); player embed from validated ids on youtube-nocookie.com
Files + SQL	job id = 32 hex chars, path stays inside `output/`; all SQL parameterised and scoped by user_id

watch out!

Known gap: login is not built yet. aq-api runs everything as the built-in local user (id 1); its `authenticate` seam is where OAuth goes. A key saved with Remember is plain text in localStorage.

★ **to remember**

→ Local-only by default: bind, host check, origin check.

→ Never trust input (regex, caps) and never echo secrets (redaction).

→ Generated HTML is sandboxed by a hash-based CSP and framing rules.

★ *recall question*

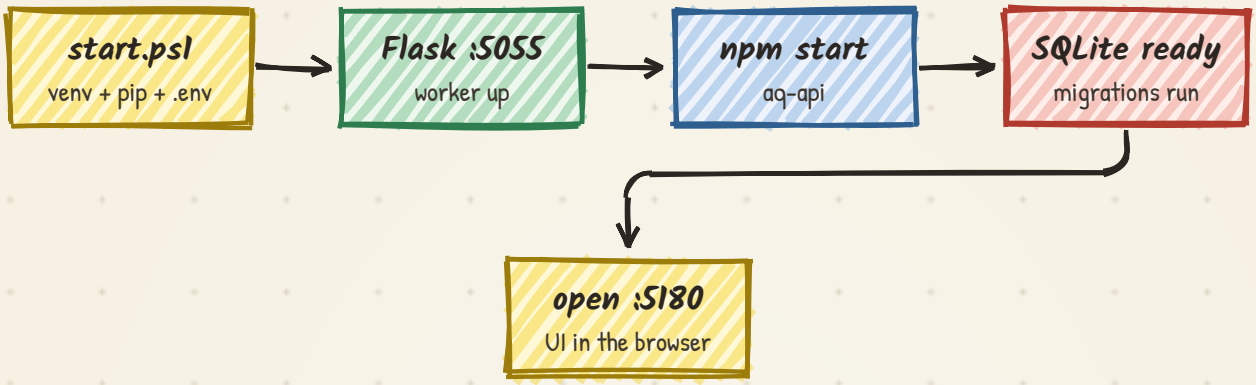
Q: How do you stop a malicious web page from using your local notes server?

A: The server answers only localhost Host headers (blocks DNS rebinding) and rejects any POST or playlist request whose Origin is not a local page. CORS allows the same set only, so another site's browser requests are refused.

next → pg. 22: run & deploy

22. run & deploy — three commands, three ports

boot sequence



worker first, then the API that depends on it

service	start	port
Python worker	`lecture-notes-ai\start.ps1`	5055
Node API + UI	`cd aq-api; npm start`	5180
UI alone (old way)	`lecture-notes-ui\serve.ps1`	5173

1. `start.ps1` creates `.venv`, installs `.[web]` only when pyproject.toml changed, opens the browser.
2. Copy `backend\.env.example` to `backend\.env` and add a Groq or Gemini key (or paste one in the UI).
3. `npm start` in aq-api opens `data/aq.db`, applies `sql/` migrations and serves the UI (`PORT`, `DB_FILE`, `PY_NOTES_URL` override defaults).
4. Open http://127.0.0.1:5180/ - never double-click index.html (a file:// page sends `Origin: null`, which is rejected).

powershell

```
# terminal 1 - AI worker
powershell -ExecutionPolicy Bypass -File D:\AQspace\lecture-notes-ai\start.ps1
# terminal 2 - app backend + UI
cd D:\AQspace\aq-api ; npm start
```

pro tip

Config: `LNA_PORT` changes the worker port; `LNAI_BROWSER` picks Edge/Chrome; `FFMPEG_PATH` points to ffmpeg for Whisper.

★ to remember

→ *Two processes to run: the Python worker (:5055) and aq-api (:5180).*

→ *Everything binds to 127.0.0.1; this is a local app, not yet a hosted deployment.*

→ *Edge or Chrome must be installed to get PDFs; ffmpeg only for caption-less videos.*

★ recall question

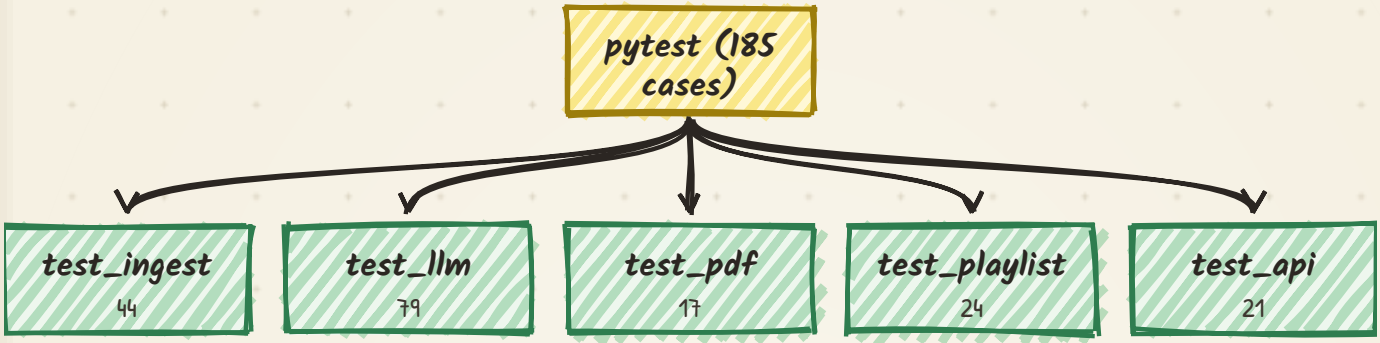
Q: What would you change to deploy this beyond a laptop?

A: Put aq-api behind a reverse proxy with HTTPS and real OAuth, move SQLite to a managed DB if multi-instance, run the Python worker with a production WSGI server and a real queue, and keep the worker on a private network.

next → pg. 23: tests & quality

23. tests & quality — everything mocked, nothing live

test pyramid



unit tests per engine module, one API suite on top

suite	covers	cases
`tests/test_ingest.py`	URL parsing, caption cleaning, fallback chain, Whisper chunks	44
`tests/test_llm.py`	JSON repair, retries, pacing, map-reduce, validation	79
`tests/test_pdf.py`	HTML rendering, diagrams, page-number check	17
`tests/test_playlist.py`	playlist id parsing, entry cleaning, limits	24
`tests/test_api.py`	Flask routes, guards, jobs, headers	21
`tests/sample_*.json`	fixture notes and raw LLM output	-
`aq-api/test/*.test.js`	playlists, progress, notes, security (node --test, in-memory DB)	32

Run `python -m pytest` from `lecture-notes-ai` (`pip install -e ".[web,dev]"`). `LNA_OFFLINE=1` skips the one live-network test. aq-api: `npm test` (`node --test`, in-memory SQLite, fake Python via injected `fetchImpl`).

pro tip

Because all provider and YouTube calls are mocked, the suite is fast and free, so it can run on every change.

★ to remember

→ *About 185 Python cases plus 32 aq-api tests; no network or API keys needed.*

→ *The server's `load_modules` is monkeypatchable, which is how test_api isolates the engine.*

→ *One optional live test is behind LNA_OFFLINE.*

★ recall question

Q: How do you test code that calls YouTube and paid LLMs?

A: Mock the boundaries: HTTP responses for providers and fake transcripts for ingest, with fixtures like sample_llm_output.json. Tests then assert parsing, retries, validation and rendering without network or keys.

next → pg. 24: limits & future scope

24. limits & future scope — *what is missing, in order*

roadmap



earliest to latest

- ***Limits today***: YouTube only; private, age-restricted and live videos fail; caption-less videos need a Groq key + ffmpeg; PDFs need Edge/Chrome; very long lectures take minutes and hit free-tier rate limits.
- ***Auth***: no login yet. Add OAuth in aq-api's middleware; make every playlist/progress row user-scoped.
- ***Sync***: today write-through with a dirty flag and last-writer-wins progress; next is per-user data and smarter multi-device conflict handling.
- ***Queue***: replace threads + memory jobs with a persistent queue so restarts keep jobs.
- ***Deploy***: HTTPS, reverse proxy, production WSGI server, hosted DB.
- ***Product***: notes search, per-video notes tab in the player, spaced-repetition from recall questions, more sources than YouTube.

note to self

Everything on this list fits the existing seams: the auth middleware, the notes cache by video_id and the stateless worker.

★ ***to remember***

→ The seams (auth middleware, video_id cache, stateless worker) are already in place.
→ Order matters: auth before multi-device, queue before hosting.
→ Biggest user-visible wins: login, notes search, revision from recall questions.

★ recall question

Q: If you had one more sprint, what would you build first?

A: Authentication in aq-api, because every other feature (per-user playlists, multi-device progress, hosted deploy) needs real user identity first, and the middleware seam already exists.

next → pg. 25: glossary, then the recap

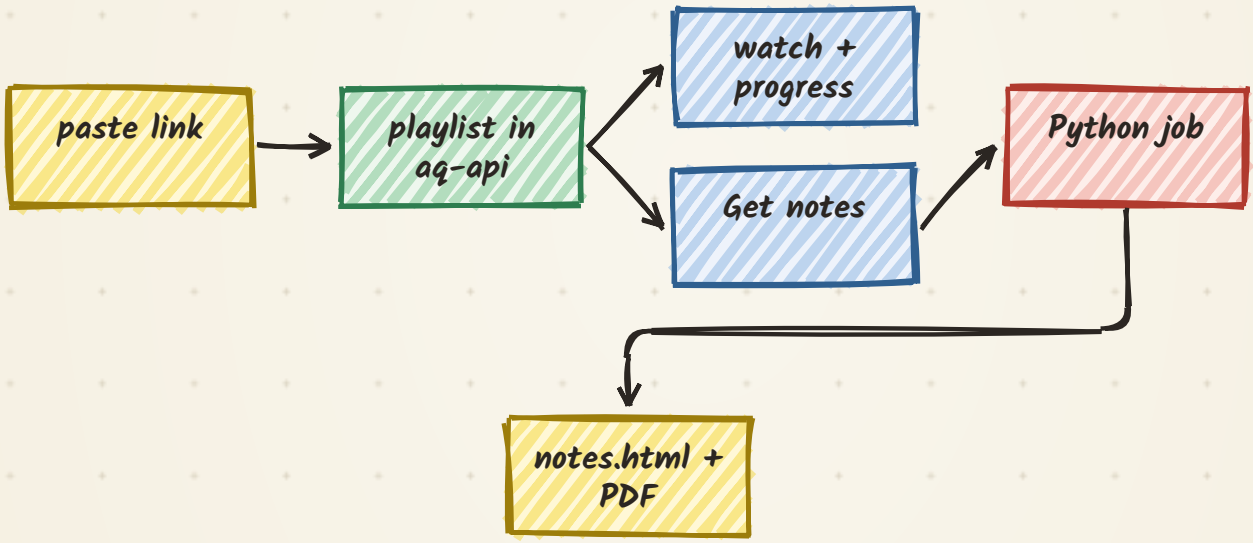
glossary — words worth knowing

aq-api	The Node/Express + SQLite application backend: users, playlists, progress, notes records; serves the UI on :5180.
worker	The Python Flask service on :5055 that makes transcripts, notes and PDFs; stateless about users.
notes_engine	The pip-installable Python package behind the CLI, the library and the web server.
job	One notes generation run in the worker, tracked in memory with stage and percent.
Notes JSON	The validated structure (meta, cover, sections, glossary, recap) that the renderer draws.
transcript	Cleaned, timed caption segments of a video, the only input the LLM sees.
map-reduce	Long-lecture strategy: digest each slice, compose sections in batches, then write cover and recap.
Whisper	Speech-to-text (Groq whisper-large-v3-turbo) used only when a video has no captions.
SSRF	Server-side request forgery: tricking the server into fetching internal URLs; blocked by strict video-id parsing.
CSP	Content-Security-Policy header deciding which scripts, frames and styles a page may use.
DNS rebinding	Attack making a web page reach localhost services; stopped by only answering local Host names.
FLIP	First-Last-Invert-Play animation technique used by split.js to morph the list into a nav.
IIFE	Immediately-invoked function expression; each UI file wraps itself in one and exports onto window.AQ.
chroma card	Playlist card: greyscale cover that reveals colour under a pointer spotlight.
watch state	Per-video {pos, dur, done, updated} stored under aq.watch.v1.
resume	Reopening a video at its saved position; button label becomes Resume m:ss.
video_id	The 11-character YouTube id; primary key for notes cache and watch progress.
OAuth	Planned login method; will plug into aq-api's auth middleware.
pacer	Client-side throttle that keeps LLM calls under the provider's tokens-per-minute limit.
recall question	The yellow Q/A box ending each notebook page, written as an interview question.

the whole picture, recapped

1. UI (vanilla JS) shows playlists, plays videos, tracks progress; it talks to one door, aq-api on :5180.
2. aq-api (Node, Express, SQLite) owns users, playlists, progress and the notes cache by video_id.
3. The Python worker (:5055) turns a video id into notes: ingest -> llm -> validate -> pdf.
4. Notes generation is a job: start, poll stage/percent, then view notes.html or download the PDF.
5. Progress is time-based: watched seconds over total seconds, derived on the fly.
6. Security is layered: localhost bind, host/origin guard, input caps, redaction, CSP.
7. Seams for the future: auth middleware, notes cache, stateless worker.

end to end



one lecture, from click to notebook